

TAIKO

Plataforma SaaS con Inteligencia Artificial para la Gestión de Catálogos y Atención al Cliente en PYMES

Trabajo de Fin de Grado Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Web

Autor: Guillermo Andújar Martínez **Tutor:** — **Centro educativo:** — **Curso académico:** 2024 – 2025

Índice

1. [Introducción](#)
 - 1.1 Contexto y motivación
 - 1.2 Objetivos del proyecto
 - 1.3 Estructura del documento
 2. [Estado del arte](#)
 - 2.1 Mercado y contexto sectorial
 - 2.2 Soluciones de referencia
 - 2.3 Análisis competitivo y oportunidad
 3. [Planificación del proyecto](#)
 - 3.1 Metodología de trabajo
 - 3.2 Fases del proyecto
 - 3.3 Recursos y herramientas utilizadas
 4. [Análisis de requisitos](#)
 - 4.1 Objetivos y requisitos funcionales
 - 4.2 Requisitos no funcionales
 - 4.3 Matriz de trazabilidad
 5. [Casos de uso](#)
 - 5.1 Actores del sistema
 - 5.2 Descripción de casos de uso
 - 5.3 Diagramas de casos de uso
 6. [Diseño del sistema](#)
 - 6.1 Arquitectura general
 - 6.2 Modelo Entidad-Relación
 - 6.3 Modelo físico de la base de datos
 - 6.4 Diseño de la API REST
 7. [Implementación](#)
 - 7.1 Backend: Spring Boot
 - 7.2 Motor de IA: RAG con pgvector
 - 7.3 Frontend: React
 - 7.4 Integración con Telegram
 - 7.5 Seguridad
 8. [Pruebas](#)
 9. [Despliegue](#)
 10. [Conclusiones y trabajo futuro](#)
 11. [Bibliografía](#)
-

1. Introducción

1.1 Contexto y motivación

La transformación digital de los pequeños y medianos comercios es uno de los grandes retos del sector comercial actual. Muchas de estas empresas carecen de las herramientas tecnológicas necesarias para competir en igualdad de condiciones con las grandes plataformas de venta digital, y cuando acceden a soluciones existentes, éstas suelen ser costosas, difíciles de configurar o poco adaptables a la naturaleza específica de cada negocio.

Al mismo tiempo, la irrupción de los modelos de lenguaje de gran escala (LLM) y las técnicas de búsqueda semántica han abierto una nueva frontera en la atención al cliente: la posibilidad de ofrecer asistentes virtuales que respondan con precisión a preguntas sobre el catálogo real de un negocio, sin alucinaciones ni respuestas genéricas, y con integración en los canales digitales que los clientes ya usan.

Este proyecto nace de la identificación de esa doble oportunidad: por un lado, la necesidad de una herramienta de gestión de catálogos accesible para PYMES; por otro, la posibilidad de integrar IA generativa de forma práctica y útil sobre esos datos de negocio. El resultado es **Taiko**, una plataforma SaaS que une ambas necesidades en una sola solución desplegada en la nube.

1.2 Objetivos del proyecto

El proyecto persigue los siguientes objetivos principales:

- **OB-001 · Gestión eficiente del catálogo:** Proporcionar a los administradores del comercio un panel de control completo para gestionar su inventario de productos: crear, editar, eliminar, buscar e importar de forma masiva desde CSV o Excel.
- **OB-002 · Atención automatizada con chatbot multicanal:** Ofrecer un asistente virtual basado en IA que responda consultas en lenguaje natural consultando el catálogo real del negocio en tiempo real, desplegable tanto en la web como en Telegram.
- **OB-003 · Gestión de usuarios y control de acceso:** Implementar un sistema completo de autenticación y autorización por roles, con gestión de usuarios desde el panel de administración.
- **OB-004 · Accesibilidad, usabilidad y seguridad:** Garantizar que la plataforma sea intuitiva para usuarios sin conocimientos técnicos, responsive para distintos dispositivos, y segura según las buenas prácticas actuales del sector.

1.3 Estructura del documento

El documento se organiza siguiendo la estructura habitual de un proyecto de ingeniería de software: tras este capítulo introductorio, se presenta el análisis del mercado y de los competidores (capítulo 2), la planificación seguida (capítulo 3), el análisis de requisitos (capítulo 4), los casos de uso (capítulo 5), el diseño técnico del sistema (capítulo 6), los detalles de la implementación (capítulo 7), las pruebas realizadas (capítulo 8), el proceso de despliegue en producción (capítulo 9) y finalmente las conclusiones y líneas de trabajo futuro (capítulo 10).

2. Estado del arte

2.1 Mercado y contexto sectorial

El comercio electrónico en España alcanzó en 2024 una facturación récord de más de **95.200 millones de euros**, representando un crecimiento del 13,1 % respecto al año anterior, según datos de la Comisión Nacional de los Mercados y la Competencia (CNMC). Solo en el cuarto trimestre de 2024 se registraron más de 478 millones de transacciones, un 10,7 % más que en el mismo período de 2023.

A nivel europeo, el comercio electrónico B2C creció un 7 % en 2024, situando a España como el **tercer mercado por volumen** tras Francia y Reino Unido. Este contexto evidencia que la digitalización de los negocios minoristas ya no es una opción estratégica sino una necesidad operativa.

En el ámbito de la atención al cliente automatizada, el avance ha sido igualmente significativo. Soluciones como Zendesk han demostrado capacidad para **automatizar hasta el 44 % de las solicitudes** de soporte, reduciendo tiempos de resolución en un 87 %. Los chatbots basados en LLM han madurado hasta el punto en que pueden gestionar consultas complejas sobre catálogos de productos sin intervención humana.

2.2 Soluciones de referencia

Solución	Categoría	Fortaleza principal	Limitación para PYMES
Shopify	Gestión de tienda	Ecosistema completo, 4,8M tiendas activas	Coste elevado, complejidad de configuración
PrestaShop	Gestión de tienda	Open source, muy extendido en Europa	Requiere servidor propio y conocimientos técnicos
Dialogflow (Google)	Chatbot	Integración multicanal, millones de interacciones/mes	Sin conexión nativa al catálogo del negocio
Zendesk	Atención al cliente	+200.000 clientes globales, automatización avanzada	Precio inaccesible para pequeños comercios
Square POS	Punto de venta	Integración inventario + ventas	Enfocado en punto físico, no en experiencia digital

2.3 Análisis competitivo y oportunidad

Ninguna de las soluciones analizadas combina, en un único producto accesible para PYMES, la **gestión del catálogo** con un **chatbot inteligente conectado a ese catálogo en tiempo real**. Shopify y PrestaShop ofrecen gestión de inventario pero sus capacidades de IA son limitadas o requieren integraciones complejas. Dialogflow ofrece IA conversacional pero no tiene acceso directo al catálogo del negocio.

Taiko cubre exactamente ese hueco: una plataforma donde el administrador gestiona su inventario y, automáticamente, ese inventario se convierte en la base de conocimiento del chatbot que atiende a sus clientes. La arquitectura RAG garantiza que el asistente nunca responde con información inventada, sino siempre con datos reales del negocio.

3. Planificación del proyecto

3.1 Metodología de trabajo

El proyecto ha seguido una metodología **iterativa e incremental**, organizando el desarrollo en sprints cortos de una semana. Esta aproximación ha permitido validar cada funcionalidad antes de avanzar a la siguiente, adaptando el alcance en función de los resultados obtenidos en cada iteración.

Se ha utilizado **Git** como sistema de control de versiones con el repositorio alojado en GitHub, realizando commits atómicos por funcionalidad e integrando todos los cambios en la rama `main` una vez validados.

3.2 Fases del proyecto

Fase	Descripción	Entregables
1. Análisis	Definición de requisitos, casos de uso, actores y modelo de datos	Documento de requisitos, diagramas de casos de uso, E/R
2. Diseño	Arquitectura del sistema, diseño de API REST, esquema de base de datos	Diagrama de arquitectura, especificación de endpoints
3. Backend	Implementación de Spring Boot: API, seguridad JWT, servicio de chatbot, embeddings	API funcional desplegada en Railway
4. Frontend	Implementación de React: páginas, componentes, integración con API	Aplicación web desplegada en Vercel
5. IA y RAG	Integración con OpenAI, generación de embeddings, búsqueda semántica, filtro LLM	Motor de búsqueda semántica funcional
6. Telegram	Integración del bot mediante webhook	Bot operativo en producción
7. Pruebas y ajustes	Pruebas funcionales, de seguridad y de usabilidad	Correcciones y mejoras
8. Documentación	Redacción del documento final, actualización del README	Documento TFG, README profesional

3.3 Recursos y herramientas utilizadas

Entorno de desarrollo

- IntelliJ IDEA (backend Java) y Visual Studio Code (frontend React)
- Postman para pruebas de la API REST
- DBeaver para gestión y consulta de la base de datos PostgreSQL

Control de versiones y despliegue

- GitHub (repositorio: `github.com/Guillegas/Taiko`)
- Railway para el despliegue del backend y la base de datos
- Vercel para el despliegue del frontend

Servicios externos

- OpenAI API (`text-embedding-3-small` y `gpt-4o-mini`)
- Telegram Bot API (BotFather + webhook)

4. Análisis de requisitos

4.1 Objetivos y requisitos funcionales

OB-001 · Gestión eficiente del catálogo de productos

Facilitar a los administradores el control total de su catálogo, permitiendo registrar, editar, eliminar, buscar y listar artículos de forma rápida e intuitiva. Las modificaciones se reflejan de forma inmediata en el sistema, incluyendo la regeneración automática de los embeddings de IA que alimentan el chatbot.

Código	Nombre	Descripción
RF-001	Registro de productos	El administrador puede crear nuevos productos introduciendo marca, modelo, precio, descripción, color, kilómetros, tipo de carrocería, transmisión, combustible, equipamiento y etiqueta ambiental.
RF-002	Modificación y eliminación de productos	El administrador edita o elimina cualquier producto existente en cualquier momento. Al actualizar un producto, el sistema regenera automáticamente su embedding de IA.
RF-003	Listado y búsqueda de productos	El catálogo permite búsqueda por texto libre, filtrado por múltiples criterios (precio, kilómetros, carrocería, transmisión, combustible, color, disponibilidad) y ordenación de resultados.
RF-004	Importación masiva de productos	El administrador puede importar cientos de productos a la vez mediante un archivo .csv o .xlsx. El sistema valida cada fila y reporta los registros procesados correctamente y los que han fallado.
RF-005	Gestión de imágenes	Cada producto admite múltiples imágenes. El sistema valida el tipo MIME y el tamaño máximo (5 MB), genera nombres únicos por UUID y marca la primera imagen como principal.

OB-002 · Atención automatizada y validada por chatbot multicanal

Ofrecer un sistema de atención al cliente automático mediante un chatbot inteligente que responda consultas sobre el catálogo en lenguaje natural, desplegable en la web y en Telegram.

Código	Nombre	Descripción
RF-006	Respuestas automáticas del chatbot	El chatbot procesa la consulta del usuario, busca los productos más relevantes mediante similitud vectorial, filtra los resultados con un segundo LLM ("juez") y genera una respuesta en lenguaje natural con recomendaciones específicas del inventario real.
RF-007	Despliegue en múltiples canales	El chatbot está disponible en la web (interfaz propia) y en Telegram (bot integrado mediante webhook).
RF-008	Almacenamiento del historial de conversaciones	Todas las conversaciones se persisten en base de datos, asociadas al usuario autenticado. Un usuario puede retomar cualquier conversación previa desde cualquier dispositivo.
RF-009	Descarga de conversaciones	El usuario puede exportar el contenido de una conversación en tres formatos: PDF (generado en el navegador), TXT y JSON estructurado (generados en el servidor).

RF-010	Acceso al historial	La sección "Mis conversaciones" muestra todas las sesiones del usuario con preview del primer mensaje, fecha y número total de mensajes. El usuario puede seleccionar cualquier conversación para reanudarla o eliminarla.
---------------	---------------------	--

OB-003 · Gestión de usuarios y control de acceso

Permitir la gestión completa de los usuarios de la plataforma, incluyendo registro, autenticación, edición y desactivación, con control de acceso granular por rol.

Código	Nombre	Descripción
RF-011	Autenticación segura	Login mediante correo electrónico y contraseña. Las contraseñas se almacenan cifradas con BCrypt. El sistema emite un JWT firmado con un secreto configurable y expiración de 24 horas.
RF-012	Registro de usuarios	Cualquier persona puede crear una cuenta de cliente desde la interfaz web. El rol <code>admin</code> solo puede asignarse desde el panel de administración.
RF-013	Edición de usuarios	El administrador puede modificar nombre, email, teléfono, rol y estado (activo/inactivo) de cualquier usuario desde el panel de administración. El propio usuario puede editar su nombre, teléfono y contraseña desde su perfil.
RF-014	Listado y filtrado de usuarios	El panel de administración muestra todos los usuarios con filtros por nombre/email, rol y estado.

OB-004 · Accesibilidad, usabilidad y seguridad (Requisitos No Funcionales)

Ver sección 4.2.

4.2 Requisitos no funcionales

Código	Nombre	Descripción
RNF-001	Rendimiento	El sistema responde a las operaciones habituales (consultas de catálogo, mensajes del chatbot) en menos de 3 segundos en el 95 % de los casos bajo condiciones normales de uso.
RNF-002	Disponibilidad	La plataforma está disponible al menos el 99 % del tiempo, garantizado por la infraestructura gestionada de Railway y Vercel.
RNF-003	Seguridad	Todos los endpoints sensibles requieren autenticación JWT. Las rutas de escritura y administración exigen el rol <code>admin</code> . Las contraseñas se almacenan con BCrypt. El CORS está configurado de forma centralizada. No se exponen stacktraces en las respuestas de error.
RNF-004	Usabilidad	La interfaz está diseñada para usuarios sin conocimientos técnicos. Incluye modo oscuro/claro, diseño responsive adaptado a móvil y escritorio, y mensajes de error comprensibles.

RNF-005	Mantenibilidad	El código está estructurado en capas (controller / service / repository), con comentarios explicativos en los puntos no evidentes, logging estructurado con SLF4J y sin dependencias circulares.
RNF-006	Portabilidad	La arquitectura separa completamente frontend y backend. El backend puede desplegarse en cualquier entorno con Java 17+ y acceso a PostgreSQL. El frontend puede servirse desde cualquier CDN.

4.3 Matriz de trazabilidad

	RF-01	RF-02	RF-03	RF-04	RF-05	RF-06	RF-07	RF-08	RF-09	RF-10	RF-11	RF-12	
OB-001	✓	✓	✓	✓	✓								
OB-002						✓	✓	✓	✓	✓			
OB-003											✓	✓	

5. Casos de uso

5.1 Actores del sistema

El sistema reconoce tres actores diferenciados:

- **Proveedor de la plataforma (Taiko):** Empresa desarrolladora responsable del mantenimiento, actualizaciones y soporte técnico de la plataforma. Tiene acceso completo al sistema a nivel de infraestructura.
- **Propietario del Comercio (Admin):** Usuario que contrata la plataforma para su negocio. Gestiona el catálogo de productos, administra usuarios, configura el chatbot y tiene acceso al panel de administración completo.
- **Cliente Final:** Consumidor que accede a la plataforma para consultar el catálogo y usar el chatbot. Puede registrarse para guardar su historial de conversaciones y exportarlas.

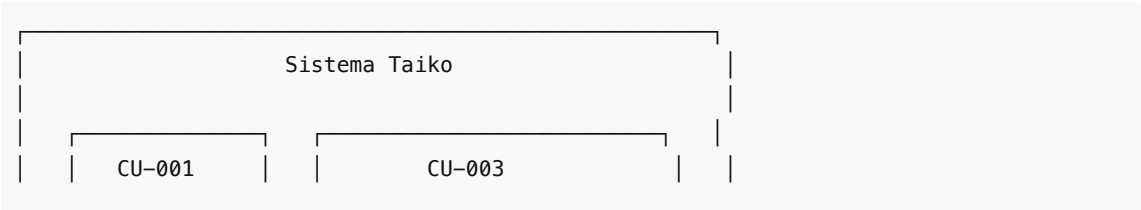
5.2 Descripción de casos de uso

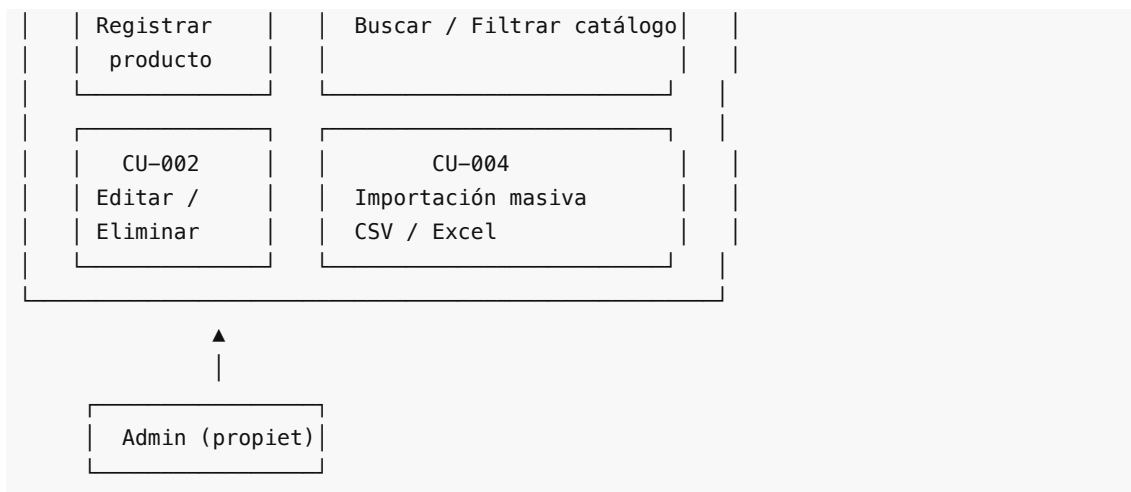
Código	Nombre	Actores	Descripción
CU-001	Registrar producto	Admin	El administrador accede al panel y crea un nuevo producto completando el formulario con todos sus atributos (nombre, precio, imágenes, relaciones con tablas auxiliares). El sistema genera automáticamente el embedding vectorial para búsqueda semántica.
CU-002	Modificar y eliminar producto	Admin	El administrador edita los campos de un producto existente o lo elimina permanentemente. En caso de

			edición, el embedding se regenera.
CU-003	Buscar y filtrar catálogo	Admin / Cliente	Cualquier usuario consulta el catálogo aplicando filtros combinados. El cliente puede además usar la búsqueda semántica por texto libre.
CU-004	Importar productos masivamente	Admin	El administrador sube un archivo .csv o .xlsx con múltiples productos. El sistema procesa cada fila, crea los vehículos y genera sus embeddings, y devuelve un resumen de éxitos y fallos.
CU-005	Consultar chatbot	Cliente	El cliente final envía un mensaje al chatbot. El sistema lo convierte a vector, busca los productos más similares en la BD, aplica un filtro LLM y responde en lenguaje natural recomendando productos del catálogo.
CU-006	Probar chatbot	Admin	El administrador utiliza la misma interfaz de chat para validar que el asistente responde correctamente antes de ponerlo en producción.
CU-007	Usar chatbot por Telegram	Cliente	El cliente inicia una conversación con el bot de Telegram. El sistema mantiene una sesión mapeada al chat_id de Telegram y usa el mismo motor de IA que la interfaz web.
CU-008	Ver historial de conversaciones	Cliente	El usuario autenticado accede a "Mis conversaciones", ve una lista ordenada cronológicamente con preview de cada sesión y puede reanudar cualquiera de ellas.
CU-009	Exportar conversación	Cliente	Desde una conversación activa, el usuario descarga su contenido en PDF, TXT o JSON.
CU-010	Registrarse / Iniciar sesión	Cliente / Admin	El usuario crea una cuenta o inicia sesión con email y contraseña. El sistema devuelve un JWT que autoriza las peticiones posteriores.
CU-011	Gestionar usuarios	Admin	El administrador consulta, filtra, edita (nombre, email, teléfono, rol, estado) y elimina usuarios desde el panel de administración.
CU-012	Editar perfil propio	Cliente	El usuario autenticado modifica su nombre, teléfono o contraseña desde la página de perfil. El cambio de contraseña requiere introducir la contraseña actual.

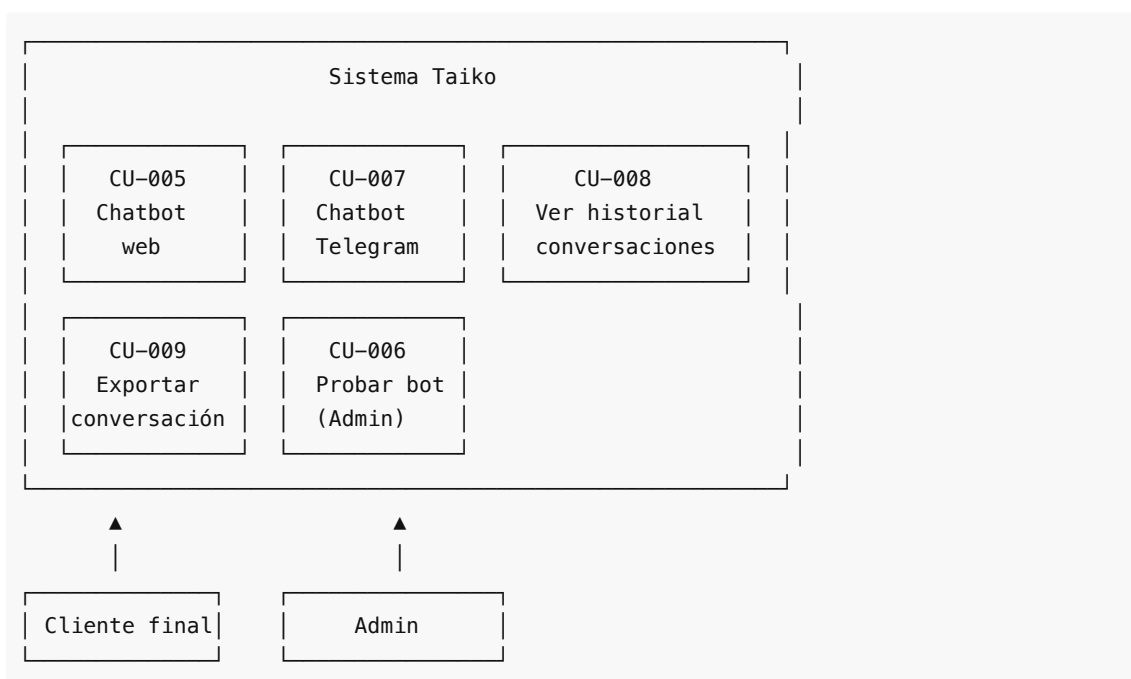
5.3 Diagramas de casos de uso

OB-001 · Gestión del catálogo

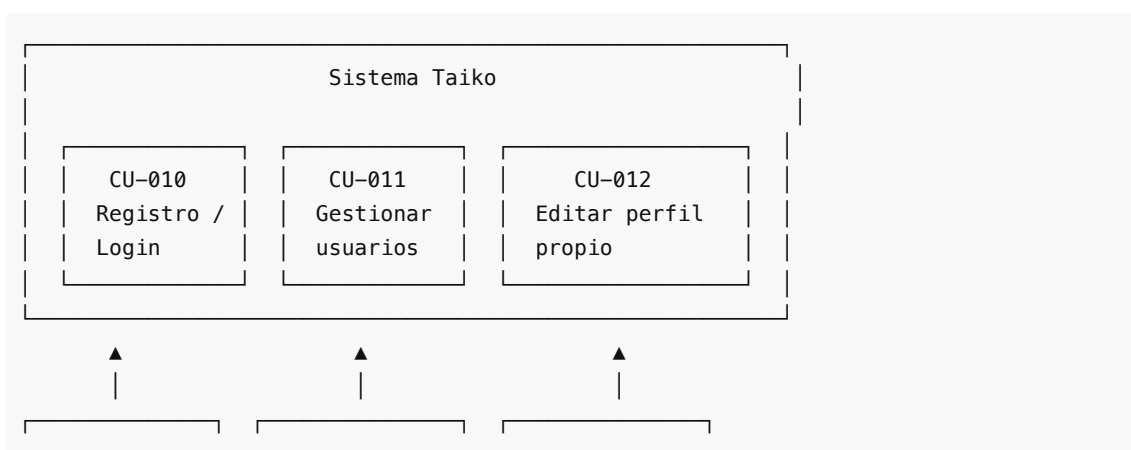




OB-002 · Chatbot multicanal



OB-003 · Gestión de usuarios

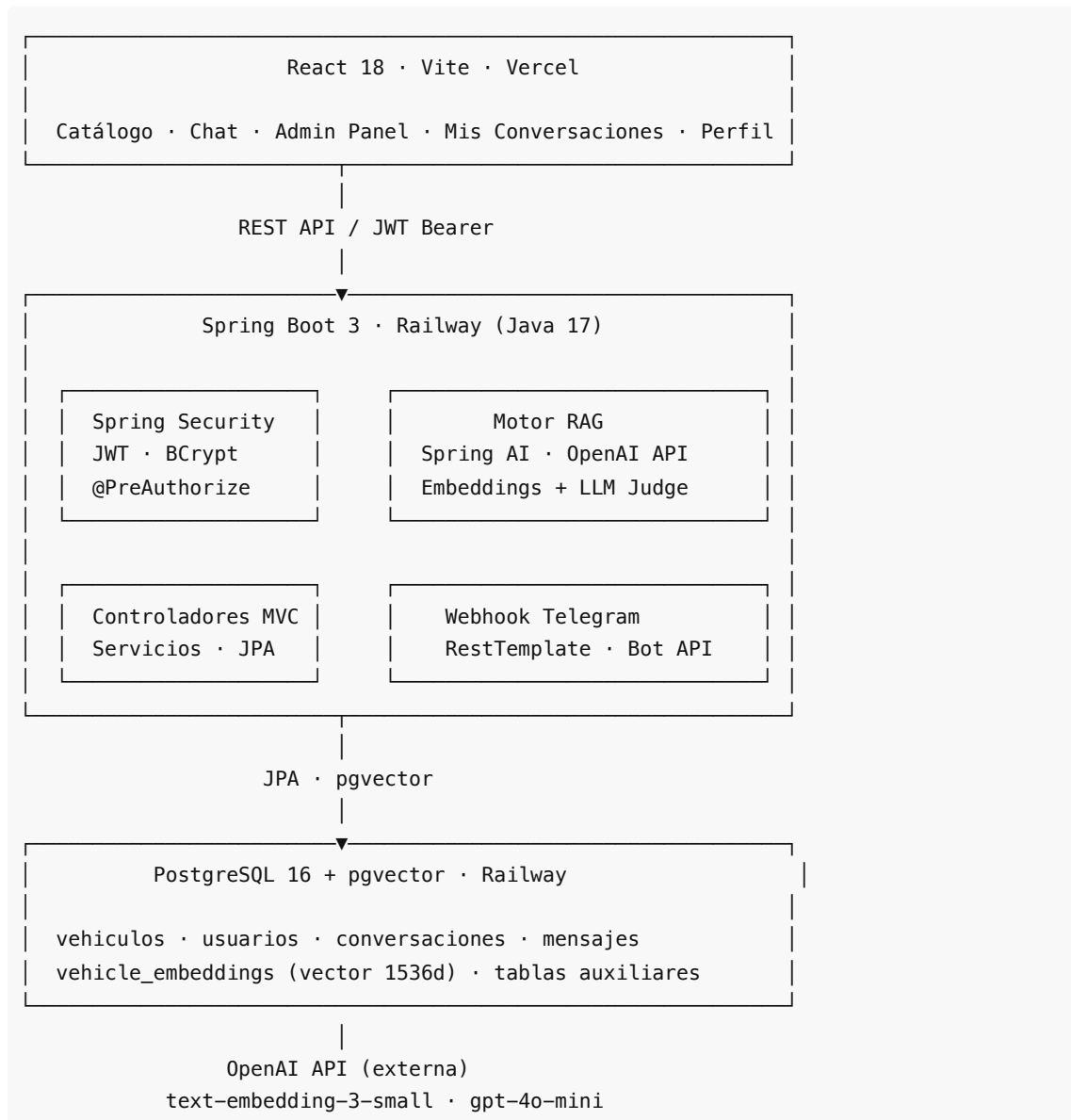


Cliente final	Admin	Cliente/Admin
---------------	-------	---------------

6. Diseño del sistema

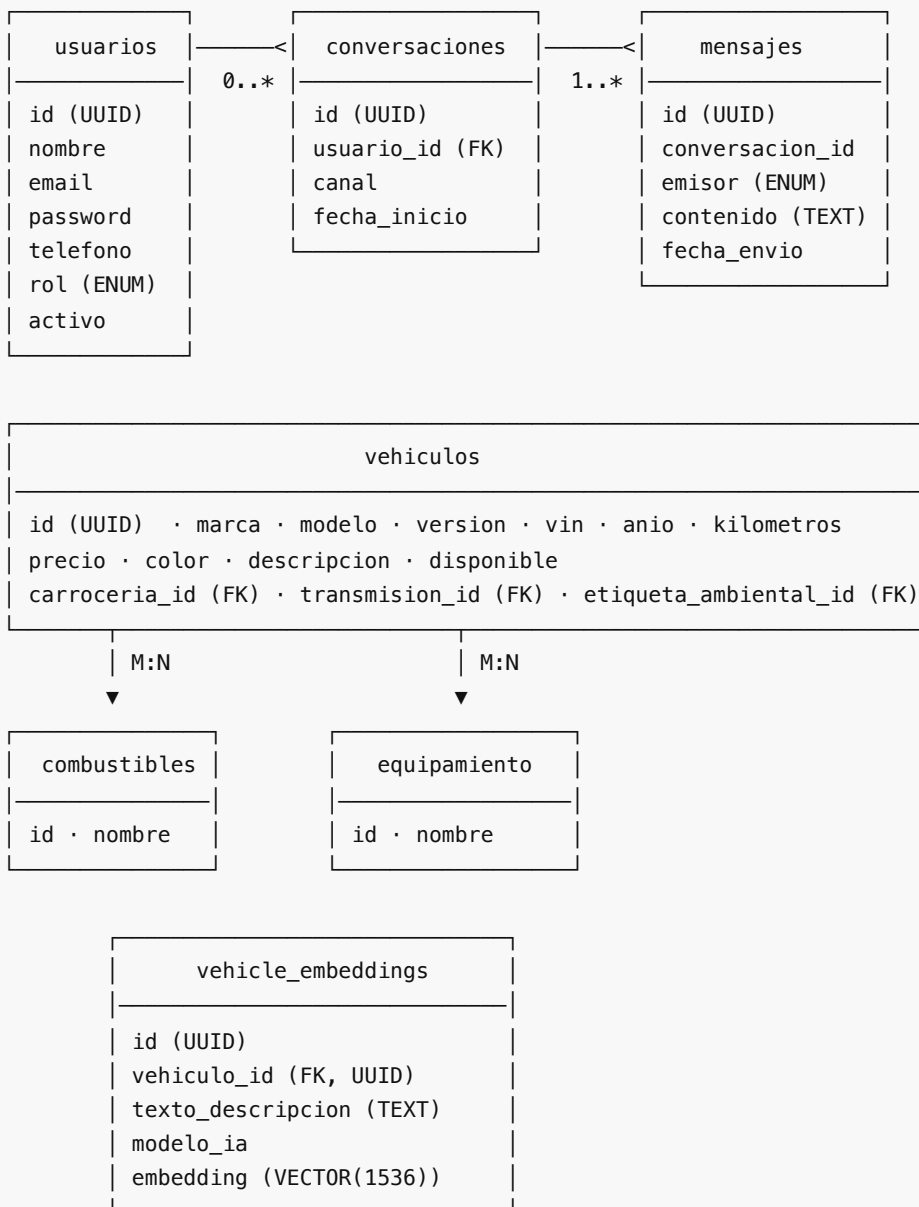
6.1 Arquitectura general

Taiko sigue una **arquitectura de tres capas** desacoplada: el frontend React se comunica con el backend Spring Boot exclusivamente a través de una API REST con autenticación JWT. El backend interactúa con PostgreSQL para la persistencia y con la API de OpenAI para la generación de embeddings y las respuestas del chatbot.



6.2 Modelo Entidad-Relación

Las entidades principales del sistema y sus relaciones son las siguientes:



Tablas auxiliares: carrocerias · transmisiones · etiquetas_ambientales
 imagenes (vehiculo_id FK, url, es_principal)

6.3 Modelo físico de la base de datos

```

-- Extensión necesaria para búsqueda vectorial
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE usuarios (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  nombre      VARCHAR(100) NOT NULL,
  email       VARCHAR(150) NOT NULL UNIQUE,
  password    VARCHAR(255) NOT NULL,
  telefono    VARCHAR(20),

```

```

rol          VARCHAR(20) NOT NULL DEFAULT 'cliente',
activo       BOOLEAN     NOT NULL DEFAULT true,
created_at   TIMESTAMP   DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE conversaciones (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  usuario_id  UUID REFERENCES usuarios(id),
  canal       VARCHAR(50)  DEFAULT 'web',
  fecha_inicio TIMESTAMP   DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE mensajes (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  conversacion_id UUID NOT NULL REFERENCES conversaciones(id),
  emisor      VARCHAR(20) NOT NULL, -- 'cliente' | 'chatbot'
  contenido   TEXT NOT NULL,
  fecha_envio  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE vehiculos (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  marca       VARCHAR(100),
  modelo      VARCHAR(100),
  version     VARCHAR(100),
  vin         VARCHAR(17) UNIQUE,
  anio        INTEGER,
  kilometros  INTEGER,
  precio      NUMERIC(10,2),
  color       VARCHAR(50),
  descripcion TEXT,
  disponible  BOOLEAN DEFAULT true,
  carroceria_id INTEGER REFERENCES carrocerias(id),
  transmision_id INTEGER REFERENCES transmisiones(id),
  etiqueta_ambiental_id INTEGER REFERENCES etiquetas_ambientales(id)
);

CREATE TABLE vehicle_embeddings (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  vehiculo_id UUID REFERENCES vehiculos(id),
  texto_descripcion TEXT,
  modelo_ia   VARCHAR(100),
  embedding   VECTOR(1536)
);

-- Índice para búsqueda por similitud coseno
CREATE INDEX ON vehicle_embeddings USING ivfflat (embedding vector_cosine_ops);

CREATE TABLE imagenes (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  vehiculo_id UUID REFERENCES vehiculos(id),
  url         VARCHAR(500),

```

```
    es_principal BOOLEAN DEFAULT false
);
```

6.4 Diseño de la API REST

La API sigue los principios REST: recursos identificados por URL, verbos HTTP semánticos, respuestas en JSON y códigos de estado HTTP estándar.

Método	Endpoint	Autenticación	Descripción
POST	/api/auth/signup	Pública	Registro de nuevo usuario
POST	/api/auth/login	Pública	Login, devuelve JWT
GET	/api/cars	Pública	Listado completo del catálogo
GET	/api/cars/{id}	Pública	Detalle de un vehículo
GET	/api/cars/search?q=	Pública	Búsqueda semántica
POST	/api/cars/new	Admin	Crear vehículo
PUT	/api/cars/{id}	Admin	Actualizar vehículo
DELETE	/api/cars/{id}	Admin	Eliminar vehículo
POST	/api/cars/{id}/images	Admin	Añadir imágenes
POST	/api/cars/import	Admin	Importación masiva CSV/Excel
POST	/api/cars/seed	Admin	Regenerar todos los embeddings
POST	/api/chat/start	Pública	Iniciar conversación
POST	/api/chat/{id}/message	Pública	Enviar mensaje al chatbot
GET	/api/chat/{id}/history	Pública	Historial de una conversación
GET	/api/chat/{id}/export/txt	Usuario	Exportar conversación como TXT
GET	/api/chat/{id}/export/json	Usuario	Exportar conversación como JSON
GET	/api/chat/mis-conversaciones	Usuario	Listado de mis conversaciones
DELETE	/api/chat/conversaciones/{id}	Usuario	Eliminar conversación
GET	/api/user/profile	Usuario	Obtener perfil propio
PUT	/api/user/profile	Usuario	Actualizar perfil propio
GET	/api/admin/users	Admin	Listar todos los usuarios
PUT	/api/admin/users/{id}	Admin	Editar usuario

DELETE	/api/admin/users/{id}	Admin	Eliminar usuario
POST	/api/upload/image	Usuario	Subir imagen al servidor
POST	/api/webhook/telegram	Pública (Telegram)	Recepción de updates del bot
GET	/api/catalogo/carrocerias	Pública	Listado de tipos de carrocería
GET	/api/catalogo/transmisiones	Pública	Listado de tipos de transmisión
GET	/api/catalogo/combustibles	Pública	Listado de combustibles
GET	/api/catalogo/equipamiento	Pública	Listado de equipamiento
GET	/api/catalogo/etiquetas-ambientales	Pública	Listado de etiquetas DGT

7. Implementación

7.1 Backend: Spring Boot

El backend se ha desarrollado con **Spring Boot 3** sobre **Java 17**, siguiendo una arquitectura en capas:

```
com.taiko.backend/
├─ config/           → SecurityConfig (CORS, JWT, rutas protegidas)
├─ controller/       → Controladores REST por recurso
├─ model/            → Entidades JPA y DTOs
├─ repository/       → Interfaces Spring Data JPA
├─ security/         → JwtUtil, JwtFilter, CustomUserDetailsService
└─ service/          → Lógica de negocio
```

Gestión de vehículos (VehículoService): El servicio centraliza toda la lógica de negocio: creación con relaciones, actualización, importación CSV/Excel y generación de embeddings. Cada operación de escritura sobre un vehículo desencadena automáticamente la regeneración de su embedding vectorial en OpenAI, garantizando que el chatbot siempre trabaja con datos actualizados.

Chatbot (ChatbotService): Implementa el flujo completo de una conversación: recibe el mensaje del usuario, persiste el mensaje en base de datos, construye el contexto para el LLM (prompt de sistema + inventario actual + historial de la sesión), llama a la API de OpenAI y persiste la respuesta. Extrae además los UUIDs de vehículos recomendados mediante un patrón de expresión regular sobre la respuesta del modelo.

7.2 Motor de IA: RAG con pgvector

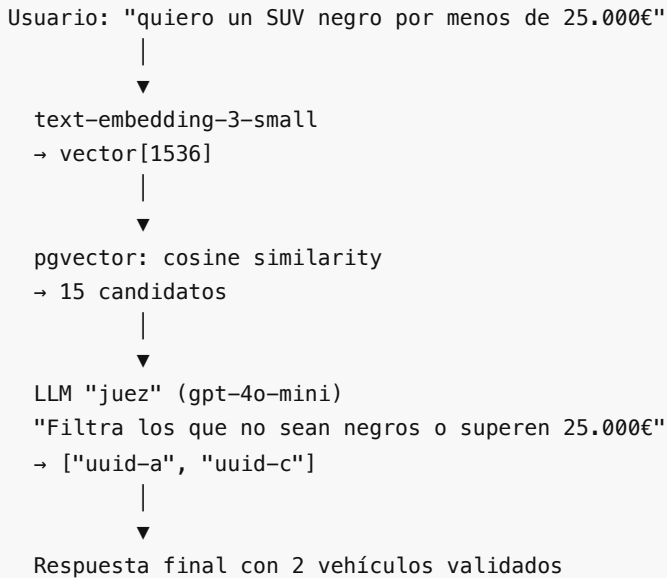
El sistema de búsqueda semántica implementa una variante del patrón **Retrieval-Augmented Generation (RAG)** en dos etapas:

Etaa 1 — Recuperación por similitud vectorial:

1. El texto de búsqueda del usuario se convierte a un vector de 1.536 dimensiones usando el modelo `text-embedding-3-small` de OpenAI.
2. Se ejecuta una consulta de similitud coseno contra la tabla `vehicle_embeddings` usando el operador `<=>` de pgvector, recuperando los 15 candidatos más cercanos.

3. La similitud se normaliza a porcentaje: $\text{similitud} = (1 - \text{distancia}/2) \times 100$.

Etap 2 — Filtro estricto con LLM ("Juez"): Los 15 candidatos se envían a un segundo LLM con un prompt específico que le pide filtrar de forma estricta los resultados que no cumplan los criterios literales del usuario (color exacto, límite de precio, etc.). El modelo devuelve únicamente los UUIDs validados en formato JSON array, eliminando falsos positivos que la búsqueda vectorial podría incluir por similitud temática pero no por coincidencia exacta de atributos.



Generación de embeddings: El texto que se vectoriza para cada vehículo incluye todos sus atributos relevantes en forma de texto descriptivo estructurado: marca, modelo, versión, carrocería, transmisión, combustible, etiqueta ambiental, precio, color, kilometraje, equipamiento y descripción libre. Esto maximiza la precisión de la búsqueda semántica ante consultas variadas.

7.3 Frontend: React

El frontend está desarrollado con **React 18** y **Vite**, organizado en páginas independientes con enrutamiento mediante **React Router v6**:

```
src/
├── components/      → Navbar, LoginModal
├── context/         → AuthContext (JWT, estado de sesión global)
├── pages/
│   ├── Home.jsx     → Landing page
│   ├── Inventory.jsx → Catálogo con filtros avanzados
│   ├── CarDetail.jsx → Detalle con galería (lightbox, miniaturas)
│   ├── Chat.jsx      → Chatbot completo con sidebar de historial
│   ├── MisConversaciones.jsx → Historial de conversaciones
│   ├── Profile.jsx   → Edición de perfil
│   └── AdminPanel.jsx → Panel de administración
└── App.jsx          → Rutas con ProtectedRoute y AdminRoute
```

AuthContext gestiona el estado de autenticación de forma global: almacena el token JWT en `localStorage`, lo inyecta en las cabeceras de todas las peticiones autenticadas y expone funciones de

login/logout al resto de la aplicación. Las rutas protegidas (`/perfil` , `/admin`) redirigen automáticamente a la raíz si el usuario no está autenticado o no tiene el rol requerido.

Página de Chat: Es la más compleja del frontend. Implementa un sidebar de conversaciones (solo para usuarios autenticados), la lógica de creación/carga/borrado de conversaciones, el envío de mensajes con indicador de escritura, la visualización de tarjetas de vehículos recomendados con enlace al detalle, y los tres formatos de exportación (PDF generado en navegador, TXT y JSON desde la API).

Galería de imágenes (CarDetail): Incluye imagen principal de 440px, navegación con flechas izquierda/derecha, contador de posición, tira de miniaturas con borde activo, lightbox a pantalla completa con navegación por teclado (`←→` y `ESC`) y tooltip de zoom al pasar el ratón.

7.4 Integración con Telegram

El bot de Telegram funciona mediante el mecanismo de **webhook**: Telegram envía un HTTP POST al endpoint `/api/webhook/telegram` del backend cada vez que un usuario envía un mensaje al bot.

El controlador gestiona:

- **Sesiones por `chat_id`** : Un `ConcurrentHashMap<Long, UUID>` mapea cada chat de Telegram con una conversación de la base de datos, manteniendo la sesión mientras el servidor esté activo.
- **Comandos especiales:** `/start` y `/nuevo` reinician la conversación.
- **Formato MarkdownV2:** La respuesta del chatbot se escapa correctamente para el formato MarkdownV2 de Telegram antes de enviarse.
- **Tarjetas de vehículos:** Si el chatbot recomienda vehículos, se envía un segundo mensaje con el listado formateado con precio, kilómetros y color.

7.5 Seguridad

La seguridad se implementa en varias capas complementarias:

Autenticación y autorización:

- Autenticación sin estado (stateless) mediante JWT firmados con HMAC-SHA256.
- Cada petición autenticada pasa por `JwtFilter` , que valida el token y carga el `SecurityContext` de Spring.
- `CustomUserDetailsService` mapea el rol del usuario (`admin` / `cliente`) a una `SimpleGrantedAuthority` , permitiendo usar `hasAuthority('admin')` en la configuración de rutas y `@PreAuthorize` en los métodos de servicio.

Configuración de rutas:

GET	<code>/api/cars/**</code>	→ Pública
POST	<code>/api/auth/**</code>	→ Pública
POST	<code>/api/cars/new</code>	→ <code>hasAuthority("admin")</code>
PUT	<code>/api/cars/**</code>	→ <code>hasAuthority("admin")</code>
DELETE	<code>/api/cars/**</code>	→ <code>hasAuthority("admin")</code>
POST	<code>/api/cars/seed</code>	→ <code>hasAuthority("admin")</code>
POST	<code>/api/cars/import</code>	→ <code>hasAuthority("admin")</code>
GET	<code>/api/admin/**</code>	→ <code>hasAuthority("admin")</code>
POST	<code>/api/upload/**</code>	→ Autenticado
GET	<code>/api/chat/*/export/**</code>	→ Autenticado

Otras medidas de seguridad:

- Contraseñas cifradas con BCrypt (factor de coste 10).
- CORS configurado centralmente en `SecurityConfig` : solo orígenes conocidos y métodos HTTP explícitos.
- Ningún controller expone `@CrossOrigin` , evitando configuraciones contradictorias.
- Validación de entrada en todos los endpoints críticos (longitud máxima de mensajes, validación de tipo MIME y tamaño en uploads).
- Los stacktraces nunca se exponen en las respuestas de error; se usa `ResponseStatusException` con mensajes controlados.
- Los administradores no pueden eliminarse ni desactivarse a sí mismos, previniendo el bloqueo accidental de la plataforma.

8. Pruebas

8.1 Pruebas funcionales

Se han realizado pruebas manuales sobre todos los flujos de usuario documentados en los casos de uso, verificando tanto el camino feliz como los casos límite y de error:

Caso de prueba	Resultado esperado	Resultado obtenido
Registro con email ya existente	Error 409 con mensaje descriptivo	✓ Correcto
Login con credenciales incorrectas	Error 401	✓ Correcto
Acceso a <code>/api/admin</code> sin token	Error 403	✓ Correcto
Acceso a <code>/api/admin</code> con token de cliente	Error 403	✓ Correcto
Creación de vehículo (admin)	Vehículo creado + embedding generado	✓ Correcto
Importación CSV con filas inválidas	Importa las válidas, reporta las fallidas	✓ Correcto
Mensaje al chatbot con criterio de precio	Respuesta con vehículos dentro del rango	✓ Correcto
Exportación de conversación (TXT/JSON)	Archivo descargado correctamente	✓ Correcto
Recarga de página en ruta React	SPA redirige correctamente (sin 404)	✓ Correcto
Cambio de contraseña con contraseña actual incorrecta	Error 400 con mensaje	✓ Correcto
Subida de imagen con tipo no permitido	Error 400 con mensaje	✓ Correcto

8.2 Pruebas de seguridad

- Se verificó que ningún endpoint de escritura puede ser llamado sin JWT válido.
- Se comprobó que un token de rol `cliente` no puede acceder a rutas `admin` .

- Se verificó que un usuario no puede eliminar conversaciones de otro usuario (validación de propiedad en la consulta).
- Se comprobó que las contraseñas no se devuelven en ninguna respuesta de la API.

8.3 Pruebas de la integración con IA

- Se realizaron consultas en lenguaje natural con criterios combinados (precio + color + tipo) verificando que el filtro LLM elimina correctamente los falsos positivos del resultado vectorial.
- Se comprobó que el fallback por similitud (umbral 65 %) funciona cuando el LLM devuelve un formato inesperado.
- Se verificó la integración completa del bot de Telegram con consultas reales al inventario.

9. Despliegue

9.1 Infraestructura

La aplicación está desplegada en un entorno cloud completamente gestionado:

Componente	Servicio	URL
Frontend (React)	Vercel	https://frontend-xi-navy-88.vercel.app
Backend (Spring Boot)	Railway	https://backend-production-85da.up.railway.app
Base de datos	Railway (PostgreSQL 16 + pgvector)	Privada (acceso interno)

9.2 Variables de entorno en producción

Variable	Servicio	Descripción
OPENAI_API_KEY	Railway	Clave de API de OpenAI
JWT_SECRET	Railway	Secreto Base64 para firma de tokens
TELEGRAM_BOT_TOKEN	Railway	Token del bot de Telegram
APP_BASE_URL	Railway	URL pública del backend (para URLs de imágenes)
PORT	Railway	Puerto del servidor (asignado automáticamente)
VITE_API_URL	Vercel	URL base de la API del backend

9.3 Proceso de despliegue

Backend: Railway detecta el repositorio de GitHub y ejecuta el build de Maven automáticamente con cada push a `main`. El `Procfile` y la configuración de puerto dinámico (`${PORT:8080}`) son compatibles con el entorno Railway.

Frontend: Vercel detecta el proyecto Vite y ejecuta `npm run build` automáticamente. El archivo `vercel.json` incluye una regla de rewrite que redirige todas las rutas al `index.html`, resolviendo el problema clásico de 404 al recargar una página en aplicaciones SPA con React Router.

```
{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
```

Webhook de Telegram: Una vez desplegado el backend, el webhook se registra con una llamada única a la API de Telegram apuntando al endpoint público:

```
POST https://api.telegram.org/bot{TOKEN}/setWebhook
{ "url": "https://backend-production-85da.up.railway.app/api/webhook/telegram" }
```

10. Conclusiones y trabajo futuro

10.1 Conclusiones

El desarrollo de Taiko ha permitido llevar a la práctica un conjunto amplio de conocimientos adquiridos a lo largo del ciclo formativo, integrándolos en un producto coherente y funcional:

- Se ha implementado una **API REST profesional** con Spring Boot, con autenticación JWT, control de acceso por roles y manejo de errores consistente.
- Se ha integrado de forma práctica la **Inteligencia Artificial generativa** en un caso de uso real de negocio, utilizando embeddings vectoriales y RAG para conectar el catálogo del comercio con el asistente virtual.
- Se ha desarrollado un **frontend moderno** con React que cubre todos los flujos de usuario relevantes, con atención al detalle en usabilidad y experiencia de usuario.
- Se ha realizado un **despliegue en producción** completamente funcional en infraestructura cloud real (Railway + Vercel), con el bot de Telegram operativo.
- Se han aplicado **buenas prácticas de seguridad** en todas las capas del sistema.

El resultado es una plataforma que demuestra que es posible construir productos con capacidades de IA avanzadas sin grandes presupuestos, utilizando herramientas open source y APIs accesibles.

10.2 Trabajo futuro

Las siguientes líneas de mejora quedan identificadas para iteraciones futuras:

- **Aplicación móvil nativa** (Android/iOS) con React Native o Kotlin, completando la visión multiplataforma original del proyecto.
- **Integración con WhatsApp Business API**, ampliando los canales del chatbot al tercero más usado en España.
- **Panel de analíticas** para el administrador: preguntas más frecuentes, productos más consultados, tasa de conversión del chatbot.
- **Multi-tenant real:** que cada comercio tenga su propio catálogo aislado dentro de la misma instancia de la plataforma, con facturación diferenciada (modelo SaaS puro).
- **Fine-tuning del modelo de embeddings** sobre conversaciones reales del sector para mejorar la precisión semántica en el dominio específico.
- **Tests automatizados:** ampliar la cobertura con tests de integración (Spring Boot Test + Testcontainers para PostgreSQL) y tests de componente en el frontend (Vitest + Testing Library).
- **Autenticación OAuth2** (Google, GitHub) como método de acceso adicional.

11. Bibliografía

1. Spring Boot Reference Documentation — <https://docs.spring.io/spring-boot/docs/current/reference/html/>

2. **Spring Security Reference** — <https://docs.spring.io/spring-security/reference/>
3. **Spring AI Reference Documentation** — <https://docs.spring.io/spring-ai/reference/>
4. **pgvector** — Open-source vector similarity search for PostgreSQL — <https://github.com/pgvector/pgvector>
5. **OpenAI API Reference** — <https://platform.openai.com/docs/api-reference>
6. **OpenAI Embeddings Guide** — <https://platform.openai.com/docs/guides/embeddings>
7. **React** — The library for web and native user interfaces — <https://react.dev/>
8. **React Router v6 Documentation** — <https://reactrouter.com/>
9. **Telegram Bot API** — <https://core.telegram.org/bots/api>
10. **Comisión Nacional de los Mercados y la Competencia (CNMC) — Informe de comercio electrónico en España 2024** — <https://www.cnmc.es/>
11. **Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.** arXiv:2005.11401 — <https://arxiv.org/abs/2005.11401>
12. **Vercel Documentation** — <https://vercel.com/docs>
13. **Railway Documentation** — <https://docs.railway.app/>
14. **OWASP Top 10 — Web Application Security Risks** — <https://owasp.org/www-project-top-ten/>

Taiko · Trabajo de Fin de Grado Guillermo Andújar Martínez · 2025

 [Demo en producción](#) · [Repositorio GitHub](#)